

Week 4 Assignment Report

Real GPU Kernel Design: GEMV

Priyanshu Gonnade

Roll Number: 24B3959

January 26, 2026

1 Overview and Motivation

The objective of Week 4 is to implement and analyze a *real-world GPU kernel* relevant to modern machine learning workloads. Unlike earlier assignments that focused on isolated CUDA concepts, this week emphasizes complete kernel design, correctness, optimization, and performance analysis.

Matrix-Vector Multiplication (GEMV) was selected as the target operation because it represents the core computation in neural network inference when the batch size is one, which is common in edge AI and low-latency systems.

The assignment is structured around the following goals:

- Implementing a correct naive GEMV CUDA kernel
- Optimizing the kernel using GPU memory hierarchy awareness
- Benchmarking against CPU and PyTorch baselines
- Studying memory reuse strategies in production GPU libraries

2 Task 1: Naive GEMV Kernel Implementation

2.1 Problem Definition

The GEMV operation computes:

$$\mathbf{y} = \mathbf{W} \times \mathbf{x}$$

where:

- $\mathbf{W} \in \mathbb{R}^{N \times K}$ is the weight matrix
- $\mathbf{x} \in \mathbb{R}^{K \times 1}$ is the input vector
- $\mathbf{y} \in \mathbb{R}^{N \times 1}$ is the output vector

All data is stored in row-major format.

2.2 Naive Kernel Design

In the naive CUDA implementation:

- Each CUDA thread computes one output element y_i
- A thread iterates over all K columns of $\mathbf{W}$
- All accesses to $\mathbf{W}$ and $\mathbf{x}$ occur from global memory

This implementation ensures correctness but results in significant redundant global memory accesses, particularly for the input vector $\mathbf{x}$.

3 Task 2: Optimized GEMV Kernel

3.1 Optimization Strategy

The performance bottleneck in the naive kernel arises from repeated global memory loads of the input vector $\mathbf{x}$. Although all threads in a block use the same vector, each thread independently fetches it from global memory.

To address this inefficiency, shared memory is used to cache the input vector:

- The input vector $\mathbf{x}$ is cooperatively loaded into shared memory
- Each block loads $\mathbf{x}$ only once from global memory
- All threads reuse the cached vector during computation

Since the vector size is small enough to fit entirely within shared memory, this optimization significantly reduces global memory traffic.

3.2 Synchronization and Correctness

A block-level synchronization barrier (`__syncthreads()`) is used after loading the vector to ensure that all threads observe a fully populated shared memory before beginning computation.

4 Code Listings

4.1 Naive GEMV Kernel

The naive GEMV kernel maps one CUDA thread to one output element. Each thread computes the dot product of one row of the weight matrix with the input vector. All accesses are performed directly from global memory, resulting in redundant loads of the input vector.

Listing 1: Naive GEMV CUDA Kernel

```
__global__ void gemv_naive(const float* A,
                          const float* x,
                          float* y,
                          int n,
                          int k) {
    int row = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < n) {
        float sum = 0.0f;
        for (int i = 0; i < k; i++) {
            sum += A[row * k + i] * x[i];
        }
        y[row] = sum;
    }
}
```

4.2 Optimized GEMV Kernel Using Shared Memory

The optimized kernel reduces redundant global memory accesses by caching the input vector in shared memory. Threads within a block cooperatively load the vector once, after which all computations reuse the cached data.

Listing 2: Optimized GEMV CUDA Kernel with Shared Memory

```
--global__ void gemv_optimized(const float* A,
                               const float* x,
                               float* y,
                               int n,
                               int k) {
    int row = blockIdx.x * blockDim.x + threadIdx.x;
    int tid = threadIdx.x;

    __shared__ float s_x[4096];

    // Load input vector into shared memory
    for (int i = tid; i < k; i += blockDim.x) {
        s_x[i] = x[i];
    }

    __syncthreads();

    if (row < n) {
        float sum = 0.0f;
        for (int i = 0; i < k; i++) {
            sum += A[row * k + i] * s_x[i];
        }
        y[row] = sum;
    }
}
```

5 Task 3: Benchmarking and Performance Comparison

5.1 Benchmark Setup

The following implementations were benchmarked:

- CPU reference implementation
- Naive CUDA GEMV kernel
- Optimized CUDA GEMV kernel

All benchmarks were executed using identical matrix dimensions and averaged across multiple runs to reduce noise.

5.2 Results

5.3 Analysis

The optimized kernel achieves a noticeable speedup over the naive version by eliminating redundant global memory accesses to the input vector. The remaining performance gap relative to highly

Table 1: GEMV Performance Comparison

Implementation	Time (ms)	Speedup
CPU Baseline	4.57	1.0x
Naive CUDA GEMV	1.69	2.70x
Optimized CUDA GEMV	1.18	3.87x

optimized libraries is attributed to the use of FP32 arithmetic and block-level synchronization overheads.

6 Task 4: Reading Production GPU Kernels

6.1 Memory Access and Reuse

State-of-the-art libraries such as *tiny-cuda-nn* and *FlashAttention* prioritize minimizing global memory traffic. These libraries aggressively reuse data in registers and shared memory, often fusing multiple operations into a single kernel to avoid intermediate writes to global memory.

6.2 Comparison with This Implementation

Compared to production kernels, this GEMV implementation:

- Uses standard FP32 precision instead of mixed precision
- Relies on block-level synchronization
- Employs simpler shared memory caching strategies

These choices favor clarity and correctness over maximum performance.

7 Conclusion

In this assignment, a production-relevant GEMV kernel was implemented and optimized using CUDA. The results demonstrate that memory access patterns, rather than compute capability, dominate performance for GEMV workloads.

By exploiting shared memory to reduce global memory traffic, a significant performance improvement was achieved. This assignment reinforces the importance of memory hierarchy awareness in GPU kernel design and sets the stage for more advanced optimizations in future work.